

Testes de Unidade com JUnit

Uma introducao

Prof. Hiago

Universidade Estadual da Paraiba (UEPB)

- 1 O que são Testes de Unidade?
- 2 Conhecendo o JUnit
- 3 Escrevendo Nosso Primeiro Teste
- 4 Os Principais Asserts
- 5 Mais Exemplos: Sistema de Heróis
- 6 Rodando os Testes
- 7 Resumo e Prática

Voce ja passou por isso?

Situacao Comum

Voce escreve um metodo, testa "na mao" rodando o programa, ve que funciona... e uma semana depois, ao mudar outra parte do codigo, aquele metodo **quebra sem voce perceber**.

Como saber, toda vez, se o seu codigo ainda funciona?

O que é um Teste de Unidade?

Definicao

Um **teste de unidade** é um pequeno pedaço de código que verifica **automaticamente** se uma única parte do seu programa (um método, por exemplo) está funcionando do jeito esperado.

- Você escreve o teste **uma vez**
- Roda quantas vezes quiser, **em segundos**
- Se algo quebrar, o teste **avisa na hora**

Analogia

Pense em um **fiscal de qualidade** numa fabrica de brinquedos: antes de cada boneco sair da caixa, ele aperta os botoes e verifica se tudo funciona. Se algo estiver errado, o boneco nao passa.

O teste de unidade e esse **fiscal automatico** do seu codigo.

Testar na Mao x Testar com Codigo

Testar na mao	Teste automatizado
Precisa repetir tudo de novo a cada mudanca	Roda tudo de novo com um clique
Facil esquecer de testar um caso	O teste sempre verifica os mesmos casos
Demora minutos	Demora segundos
Depende da atencao da pessoa	Nao se cansa nem se distrai

O que é o JUnit?

Definicao

JUnit é o framework mais usado para escrever e rodar testes de unidade em Java.

Com o JUnit, cada teste é um **metodo Java comum**, marcado com uma anotacao especial: a `@Test`.

Vamos aprender, passo a passo, como escrever nosso primeiro teste!

O que faz a @Test

A anotacao @Test diz ao JUnit: *"este metodo aqui e um teste, execute-o e me diga se passou ou falhou"*.

Dica

Sem a @Test, o JUnit **nem sabe que aquele metodo existe** como teste – ele seria apenas um metodo comum, ignorado pelos testes.

Como deve ser um metodo de teste

- Deve ser marcado com `@Test` logo acima
- Deve ser `public`
- Deve retornar `void` (nao retorna nada)
- Nao recebe parametros
- Deve ter um **nome descritivo**, dizendo o que esta sendo testado

A Forma de um Metodo de Teste

```
1 @Test
2 public void nomeDescritivoDoTeste() {
3     // 1. Preparar os dados
4     // 2. Executar o metodo que quero testar
5     // 3. Verificar se o resultado e o esperado
6 }
```

Dica

Um bom nome de teste conta uma historia: `testSomarDoisNumerosPositivos`, `testDividirPorZeroLancaErro...`

A Classe que Vamos Testar: Calculadora

```
1 public class Calculadora {  
2  
3     public int somar(int a, int b) {  
4         return a + b;  
5     }  
6  
7     public boolean ehPositivo(int numero) {  
8         return numero > 0;  
9     }  
10 }
```

Criando a Classe de Teste

```
1 import org.junit.jupiter.api.Test;
2 import static org.junit.jupiter.api.Assertions.*;
3
4 public class CalculadoraTest {
5
6     // Nossos metodos de teste vao aqui dentro
7
8 }
```

Dica

Por convencao, a classe de teste tem o **mesmo nome da classe original**, com Test no final: Calculadora → CalculadoraTest.

Nosso Primeiro Teste: passo a passo

```
1 @Test
2 public void testSomar() {
3     Calculadora calc = new Calculadora();           // 1. Preparar
4
5     int resultado = calc.somar(2, 3);                // 2. Executar
6
7     assertEquals(5, resultado);                      // 3. Verificar
8 }
```

Exemplo

Preparamos a calculadora, executamos somar(2, 3) e verificamos se o resultado é realmente 5.

O que é um *Assert*?

Definicao

Um **assert** (afirmação) é o comando que realmente **verifica** se o resultado do seu código é o esperado. É ele quem decide se o teste passa ou falha.

Sem um assert, o teste roda mas **nunca falha** – ele não está verificando nada de verdade!

assertEquals: o mais usado

assertEquals

Verifica se dois valores são **iguais**.

```
assertEquals(esperado, obtido);
```

```
1 @Test
2 public void testSomar() {
3     int resultado = new Calculadora().somar(2, 3);
4     assertEquals(5, resultado);
5     // "Eu espero 5. O resultado obtido foi 5?"
6 }
```

Atencao

Em assertEquals, o **primeiro valor** e sempre o **esperado** e o **segundo** e o **obtido** pelo seu codigo.

```
assertEquals(esperado, obtido)
```

Trocar a ordem nao muda se o teste passa, mas muda a **mensagem de erro** exibida quando ele falha – e isso ajuda (ou atrapalha) muito na hora de descobrir o problema!

assertTrue e assertFalse

assertTrue / assertFalse

Verificam se uma condicao e **verdadeira** ou **falsa**.

```
assertTrue(condicao);    assertFalse(condicao);
```

```
1  @Test
2  public void testEhPositivo() {
3      boolean resultado = new Calculadora().ehPositivo(10);
4      assertTrue(resultado);
5  }
6
7  @Test
8  public void testEhPositivoComNumeroNegativo() {
9      boolean resultado = new Calculadora().ehPositivo(-5);
10     assertFalse(resultado);
11 }
```

Assert	O que verifica
<code>assertNull(obj)</code>	Se o objeto é <code>null</code>
<code>assertNotNull(obj)</code>	Se o objeto nao é <code>null</code>
<code>assertNotEquals(a, b)</code>	Se os valores são diferentes

Dica

Todos seguem a mesma ideia: **descrever o que voce espera** e deixar o JUnit verificar.

Praticando com um Exemplo Maior

Vamos aplicar os mesmos asserts em um sistema um pouco maior: o ControladorSistemaHerois, que cadastra e consulta herois.

```
1 public boolean cadastrarHeroiFisico(int codigo,
2     String nomeVerdadeiro, String codinome,
3     int energia, int experiencia, int nivelForca) { ... }
4
5 public Heroi buscarHeroi(int codigo) { ... }
6
7 public int contarHerois() { ... }
```

Dica

Repare: os metodos sao parecidos com os da Calculadora – recebem dados e devolvem um resultado (boolean, um objeto ou um int).

Testando um Cadastro com assertTrue

```
1 @Test
2 public void testCadastrarHeroiFisico() {
3     ControladorSistemaHerois controlador =
4         new ControladorSistemaHerois();
5
6     boolean sucesso = controlador.cadastrarHeroiFisico(
7         1, "Bruce Wayne", "Homem-Morcego", 100, 0, 85);
8
9     assertTrue(sucesso);
10 }
```

Exemplo

Cadastramos um heroi valido e esperamos que o metodo retorne true, indicando sucesso.

Testando uma Busca com assertNotNull

```
1 @Test
2 public void testBuscarHeroiCadastrado() {
3     ControladorSistemaHerois controlador =
4         new ControladorSistemaHerois();
5     controlador.cadastrarHeroiFisico(
6         1, "Bruce Wayne", "Homem-Morcego", 100, 0, 85);
7
8     Heroi heroi = controlador.buscarHeroi(1);
9
10    assertNotNull(heroi);
11 }
```

Exemplo

Depois de cadastrar o heroi de codigo 1, buscar por esse mesmo codigo **nao pode** devolver null.

Testando o Caso Contrario com assertNull

```
1 @Test
2 public void testBuscarHeroiNaoCadastrado() {
3     ControladorSistemaHerois controlador =
4         new ControladorSistemaHerois();
5
6     Heroi heroi = controlador.buscarHeroi(999);
7
8     assertNull(heroi);
9 }
```

Exemplo

Como nenhum heroi foi cadastrado com o codigo 999, a busca deve devolver null.

Testando uma Contagem com assertEquals

```
1 @Test
2 public void testContarHerois() {
3     ControladorSistemaHerois controlador =
4         new ControladorSistemaHerois();
5     controlador.cadastrarHeroiFisico(
6         1, "Bruce Wayne", "Homem-Morcego", 100, 0, 85);
7     controlador.cadastrarHeroiTecnologico(
8         2, "Tony Stark", "Homem de Ferro", 100, 0,
9         "Armadura", 90);
10
11     int total = controlador.contarHerois();
12
13     assertEquals(2, total);
14 }
```

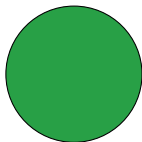
O que Mudou (ou Nao Mudou)?

Reparou a semelhanca?

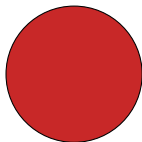
- Todo teste continua: **preparar, executar, verificar**
- `assertTrue` para retornos booleanos de sucesso
- `assertNotNull` / `assertNull` para buscas
- `assertEquals` para contagens e calculos

O sistema fica mais complexo, mas a **forma de testar** continua exatamente a mesma!

Verde ou Vermelho?



Teste passou



Teste falhou

Ao rodar os testes, cada metodo marcado com `@Test` aparece em **verde** (passou) ou **vermelho** (falhou).

Um teste vermelho nao e um bug seu na hora!

Um teste falhar significa apenas que o **resultado obtido** foi diferente do **esperado**. Isso pode ser:

- Um erro real no codigo (otimo, o teste te avisou!)
- Um erro no proprio teste (o valor esperado esta errado)

Cuidados

- Esquecer a anotacao @Test (o metodo nunca roda como teste)
- Escrever um teste sem nenhum assert (ele nunca falha, mesmo com o codigo errado)
- Colocar o valor **obtido** no lugar do **esperado** em assertEquals
- Dar nomes genericos aos testes, como teste1, teste2

Antes de terminar seu teste, confira

- Tem a anotacao @Test?
- O nome do metodo explica o que esta sendo testado?
- Tem pelo menos um assert?
- O valor esperado esta correto?
- O teste ficou **verde** ao rodar?

O que vimos hoje

- Testes de unidade verificam **automaticamente** se o código funciona
- A anotação `@Test` marca um método como teste
- Um método de teste é `public void`, sem parâmetros e com nome descritivo
- `assertEquals(esperado, obtido)` compara dois valores
- `assertTrue` / `assertFalse` verificam condições
- Verde = passou, vermelho = falhou